

Developing Full Stack Next.js Web Applications

Dr. Jose Annunziato

Chapter 1 - Building Next.js User Interfaces with HTML

The foundation of our modern digital landscape is the **Internet**, a global array of interconnected computer networks. It originated in the early 1960s as a research project commissioned by the **Advanced Research Projects Agency (ARPA)**, the research arm of the **United States Department of Defense**. The initial goal was to build **ARPANET**, a robust, decentralized, and redundant communication infrastructure capable of maintaining connectivity even in the event of major disruptions. By the 1980s, the standardization of protocols like **TCP/IP** allowed these isolated military and academic networks to proliferate and connect worldwide, establishing the underlying network of networks upon which modern digital communication relies.

The **World Wide Web** was invented in 1989 by British computer scientist **Sir Tim Berners-Lee** during his tenure at **CERN**, the European particle physics laboratory in Switzerland. The objective was to enable the efficient sharing and linking of research documents over the existing Internet. In March 1989, Sir Tim Berners-Lee proposed a system of hypertext documents connected via hyperlinks, which users access through **Uniform Resource Locators (URLs)** using the **HyperText Transfer Protocol (HTTP)**. By 1990, he had created the first **web browser** (named **WorldWideWeb**), a **web server**, and the foundational **HyperText Markup Language (HTML)** language to create HTTP **web pages**. Browsers and servers connect to one another over the internet in a **client-server** architecture. Sir Tim Berners-Lee made the Web public in 1991, and on April 30, 1993, CERN released the technology into the public domain, facilitating explosive growth. Presently, billions of static and dynamic pages power various systems ranging from simple sites to complex applications.

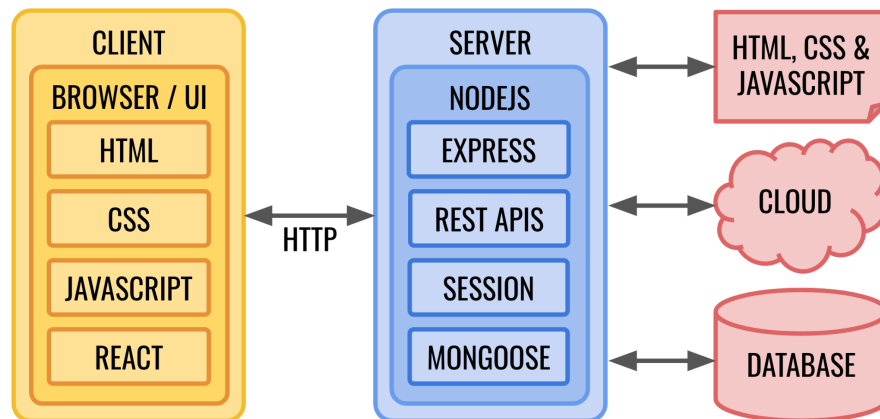


Figure 1.1 - The Client Server Architecture

Web pages are comprised of plain text documents formatted with **HTML**, a dialect of **XML (eXtensible Markup Language)**. **HTML** is a computer language utilized to format the content displayed in web pages, including properties such as foreground and background color, white spaces, text alignment, font, lists, tables, and forms. Browsers establish a network connection with servers to send **HTTP Requests** for HTML documents. These requests rely on **Uniform Resource Locators (URLs)**, which specify the server's IP address or hostname alongside the precise path to the requested document. Servers locate the requested documents, and then **respond** with the document's content. Browsers parse HTML documents to create in-memory object representations known as the **DOM (Document Object Model)**. The **DOM** consists of a hierarchical data structure where each node is configured to render content in a specific format and style. This chapter examines using HTML for formatting web pages and creating user interfaces.

JavaScript is a programming language that initially was mostly used to write programs that can execute within browsers and to programmatically manipulate the **DOM** and control what a browser renders on the screen.

JavaScript files are referenced by HTML documents, downloaded from servers, and executed within the browser to implement dynamic user interfaces. Today JavaScript has outgrown beyond the browser and is widely used for implementing general-purpose programs. **TypeScript** is a version of JavaScript developed by Microsoft, that adds strong typing support and is quickly becoming the preferred programming language for web development. Later chapters discuss programming server-side logic, including API routes and database interactions with **MongoDB**.

React is a popular JavaScript library developed by Meta for building dynamic and interactive user interfaces. It promotes a component-based architecture where developers break down complex UIs into small and reusable pieces of code called **components**. React efficiently manages the state of these components, ensuring that the user interface stays in sync with underlying data changes. Developers use React to build **Single Page Applications (SPAs)**, which provide a seamless, fluid user experience by updating only the necessary parts of the page without requiring full-page reloads. Its declarative nature simplifies the process of creating complex web interfaces by describing what the UI should look like for a given state.

Next.js is a powerful framework designed to simplify the construction of full-stack web applications. Built on top of React, it provides an all-in-one solution that includes robust features such as **server-side rendering (SSR)**, **static site generation (SSG)**, and seamless API endpoint integration. Next.js extends React by offering built-in routing, data fetching, and performance optimizations, allowing developers to create highly scalable applications that interact efficiently with backend resources, such as MongoDB, over HTTP.

This chapter describes how to install and configure a local development environment for building Next.js applications. Development is done in the local environment and then shared in a remote **GitHub** source repository. The source in **GitHub** is then deployed to a remote server hosted on **Vercel** which is optimized for Next.js and provides seamless serverless deployment. This chapter introduces creating a Next.js application and explores building user interfaces using HTML and JavaScript. Various HTML elements are described to render user interface content, such as headings, paragraphs, lists, tables, and form elements. All sections in this chapter contain exercises that introduce basic HTML elements and concepts, giving an opportunity to learn and practice HTML skills. The exercises provide detailed instructions to successfully accomplish the tasks. Make sure to complete all exercises described in the book.

The **Kambaz** sections in each chapter contain exercises that ask readers to build a fully functional web application inspired by a popular **Learning Management System (LMS)** with a similar name. The exercises provide sample code and requirements but deliberately leave out steps where the reader is expected to experiment and discover how to implement the requirements using the skills learned in prior sections. This chapter focuses on using plain **HTML** within Next.js components to implement a draft, rough prototype of various **Kambaz** screens, which at first won't look like the target screenshots. Later chapters will continue working on the **Kambaz** application, introducing **Cascade Style Sheets (CSS)** to style the Web pages so they look more like the screen shots provided, and integrating MongoDB for data persistence.

1.1 Learning Objectives

By the end of this chapter, you will be able to:

- Understand the fundamentals of HTML and how it structures web content.
- Set up a development environment for Next.js applications.
- Create and organize Next.js components using JSX.
- Use Chrome DevTools to inspect and manipulate the DOM.
- Implement various HTML elements such as headings, paragraphs, lists, tables, and images.
- Build interactive web forms with different input types.
- Use CSS to style web pages and improve user interfaces.
- Implement navigation in a Next.js single-page application (SPA) using built-in routing.

- Develop a structured approach to building user interfaces in Next.js, including an introduction to server-side features.

1.2 Setting Up the Development Environment

This section introduces **HTML** within the context of Next.js applications. A development environment is established to enable practicing **HTML** exercises within Next.js components. Examples are to be copied into the IDE as instructed, and the rendering and behavior of the code must be confirmed. Implementation of all exercises in the presented order is required, as subsequent exercises assume the completion of preceding ones.

1.2.1 Installing Node.js

Node.js is a JavaScript runtime that allows executing JavaScript from a computer console. It is essential for developing Next.js applications, as it powers the local development server, handles dependencies via **npm (Node Package Manager)**, and enables server-side features like API routes. Navigate to <https://nodejs.org/>, download the latest **LTS (Long Term Support)** version of Node.js for your operating system (recommended: version 22.x or later as of 2025), and install it on your local computer. Restart your computer if prompted, and confirm Node.js is installed by typing the command **node -v** in a console or terminal. The output should display the installed version (e.g., v22.4.0). Your version might vary slightly, but ensure it is at least 18.18 or later, as required by Next.js.

```
node -v
v22.4.0
```

Installing Node.js adds the **npm** and **npx** build tools to run, test, and package **Node.js** projects, similar to build tools such as **mvn** for **Java** and **pip** for **Python** projects. Later chapters describe how to create **HTTP Servers** with Node.js to implement **RESTful Web APIs** and integrate databases such as **MongoDB**. For now Node.js will be used to create and host a React user interface.

1.2.2 Installing an Integrated Development Environment (IDE)

While you can use any text editor, **Visual Studio Code (VS Code)** is highly recommended for Next.js development due to its excellent support for JavaScript, React, and extensions like the official Next.js extension for debugging and **IntelliSense**. Download and install VS Code from <https://code.visualstudio.com>. Once installed, open VS Code and install useful extensions such as "**ESLint**" for code linting, "**Prettier**" for formatting, and "**React Developer Tools**" for enhanced React support. Open a terminal within VS Code (via Terminal > New Terminal) to run commands seamlessly.

1.2.3 Creating a Next.js Application

React has become one of the most popular **JavaScript** libraries for building Web user interfaces. Using **npx**, a **Node.js** tool part of the **Node.js** installation, create a **React** project where we will be learning all about Web development. From the **home** directory (~) of your computer, create a directory for this semester, and then another directory under that for this course. Below are examples of creating directories from the home directory of the file system. On **macOS**, start the **Terminal** application. On a **Windows OS**, start the **console** application or **PowerShell**. On either OS, type the following to create the directory for the current year and term, e.g., ~/2049/winter/webdev.

```
cd ~           # navigate to your home directory in your file system
mkdir 2049     # creates a directory for the current year, e.g., year 2049
mkdir 2049/winter # creates a directory for the current term, e.g., winter inside 2049
```

```
mkdir 2049/winter/webdev # creates a directory called webdev inside winter/2049
cd    2049/winter/webdev # navigates to the directory webdev inside winter/2049
```

Feel free to choose other places in the file system making sure all directory and file names are all lowercase, do not have spaces in them, and are inside directories that also meet these criteria. With Node.js installed, you can create a new Next.js project using the **create-next-app** tool, which sets up a boilerplate app with best practices. From the terminal (or VS Code's integrated terminal) run the following command:

```
npx create-next-app@latest
```

If this is the first time creating a **Next.js** project, you'll be asked to install the **create-next-app** package. If this is the case, accept to proceed.

```
Need to install the following packages:
create-next-app@15.3.5
Ok to proceed? (y)
```

Name the project **kambaz-next-js**.

```
? What is your project named? > kambaz-next-js
```

Choose all the defaults

```
✓ What is your project named? ... kambaz-next-js
✓ Would you like to use TypeScript? ... No / Yes
✓ Would you like to use ESLint? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like your code inside a `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to use Turbopack for `next dev`? ... No / Yes
✓ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
Creating a new Next.js app in /users/jannunzi/neu/nextjs/kambaz-next-js.
```

Wait for the project to install.

```
Using npm.

Initializing project with template: app-tw

Installing dependencies:
- react
- react-dom
- next

Installing devDependencies:
- typescript
- @types/node
- @types/react
- @types/react-dom
- @tailwindcss/postcss
- tailwindcss
- eslint
- eslint-config-next
- @eslint/eslintrc

added 339 packages, and audited 340 packages in 36s

137 packages are looking for funding
```

```
run `npm fund` for details

found 0 vulnerabilities
Initialized a git repository.

Success! Created kambaz-next-js at /users/jannunzi/neu/nextjs/kambaz-next-js
```

Next.js will create the project in a directory called **kambaz-next-js**. **Change directory (cd)** to the project directory and run the project using `npm run dev` as shown below.

```
cd kambaz-next-js
npm run dev

> kambaz-next-js@0.1.0 dev
> next dev --turbo

  ▲ Next.js 15.3.5 (Turbopack)
  - Local:      http://localhost:3000
  - Network:    http://192.168.1.177:3000

✓ Starting...
✓ Ready in 845ms
```

To see the Web application running, use Google Chrome to navigate to the URL displayed on the console, e.g., <http://localhost:3000>. Confirm the **NEXT.js** logo render the browser as shown below. Stop the Web application by typing `Ctrl+C`. Although the **Next.js** application can be run from a terminal, prefer running it from within an IDE such as **Visual Studio Code (code)**. Using **code**, open the **kambaz-next-js** directory and start the application from the terminal window at the bottom of **code**. If the terminal is not already showing, display it from the menu **View, Terminal**. Although other browsers and IDEs are acceptable, prefer using **Google Chrome** and **Visual Studio Code**.



Figure 1.2 - Default Next.js project running in a browser

1.2.4 Creating Next.js Pages with React Components

React is a **JavaScript** library for building dynamic **web user interfaces (UI)**. In Next.js, which is built on React, user interfaces are created using **JavaScript** (or **TypeScript**) functions or classes called **components**. These components compute and return the content of the user interface as HTML-like code snippets, often incorporating logic for interactivity, data fetching, or state management. The syntax that combines JavaScript and HTML is called **JSX (JavaScript XML)**, and files typically use the **.jsx** extension. JSX allows you to seamlessly embed HTML tags within JavaScript code, making it intuitive to describe UI structures programmatically.

For enhanced type safety and scalability—especially in larger applications—**TypeScript** is recommended. TypeScript is a superset of JavaScript that adds static typing, helping catch errors early during development. Next.js supports TypeScript out-of-the-box, and files use the **.tsx** extension (as shown in examples below). Throughout this course, we'll use TypeScript to leverage its benefits in a full-stack context.

In your VS Code **EXPLORER** window on the left, navigate to the **app** directory and do all work inside the **app** directory. To practice JSX syntax in Next.js, create a new page component for Lab 1 in a file **app/labs/lab1/page.tsx** containing the source code below:

app/Labs/Lab1/page.tsx

<http://localhost:3000/Labs/Lab1>

```
export default function Lab1() {  
  return (  
    <div id="wd-lab1">  
      <h2>Lab 1</h2>  
    </div>  
  );  
};
```

Lab 1

The **Lab1** function above defines a default-exported React component, which Next.js treats as a page when placed in a folder like **app/labs/lab1/page.tsx**. The function returns an **HTML div element (division element)** that contains an **h2 heading element** with the text **Lab 1**. The **HTML** code snippet will render a heading of size 2 introducing the topic of the first chapter. The file-system routing automatically makes this accessible at **/labs/lab1** in your app. Save the file and ensure your development server is running (npm run dev). Confirm you can navigate to the **Lab1** page by pointing your browser to <http://localhost:3000/labs/lab1>.

The Next.js project uses a default styling based on Tailwind.css which we'll discuss in the next chapter. For now, comment out the global **cascading style sheet (CSS)** so we can see the raw, default style of the Web pages implemented in this course. Later chapters will discuss how to style Web pages with **CSS**. In **app/layout.tsx**, comment out the import statement as shown below

app/Layout.tsx

```
import type { Metadata } from "next";  
import { Geist, Geist_Mono } from "next/font/google";  
// import "../globals.css";  
...  
// comment out this line  
// Leave the rest of this file alone
```

1.3 Introduction to HTML

HTML (HyperText Markup Language) is a specialized dialect of **XML (eXtensible Markup Language)** designed for structuring and formatting plain text content so that web browsers can interpret and render it with specific styles, layouts, and interactivity. In Next.js, HTML elements are embedded within JSX (JavaScript XML) inside React components, allowing developers to define user interfaces declaratively while leveraging JavaScript for dynamic behavior. Consider the following HTML snippet, which styles the text "Labs" as a level-1 heading, rendering it in large, bold letters:

```
<h1>Labs</h1>
```

In code, **<h1>** and **</h1>** are called **tags**: The **<h1>** is the **opening tag**, and **</h1>** is the **closing tag**. The text "Labs" between them is the **body** or content of the tag. Tags add semantic meaning to the content—for example, h1 indicates a top-level heading, which browsers interpret by applying default styles like larger font size and bold weight. When a browser parses this HTML (or JSX in a Next.js component), it creates an in-memory representation called the **DOM (Document Object Model)**—a hierarchical tree structure where each tag becomes a

node. The DOM determines how the browser renders the page on the screen, and JavaScript (including React in Next.js) can manipulate it dynamically for updates without full page reloads. We often use the terms "tag" and "element" interchangeably, but there's a subtle distinction: A **tag** refers to the textual syntax in your code (e.g., `<h1>`), while an **element** is the broader concept, encompassing the tag, its attributes, body, and the resulting DOM node. For practical purposes in this course, either term works, but understanding this helps when debugging with browser tools like **Chrome DevTools**.

1.3.1 Structuring Web Content with the HTML Heading and Div Tags

Text documents are often broken up into several sections and subsections. Each section is usually prefaced with a short title or **heading** that summarizes the topic of the section it precedes. The font of section headings is usually larger and bolder than the plain text. HTML **heading tags** are used to format plain text so that it renders as prominent titles. There are six heading tags for different sizes: **h1**, **h2**, **h3**, **h4**, **h5**, and **h6**. Tag **h1** represents the largest heading, while **h6** is the smallest.

Another commonly used HTML element is the **div** tag (division tag), which is a generic container used to group elements together. Unlike heading tags, **div** tags do not inherently carry any specific visual formatting other than adding vertical spacing like a new line. They serve as a mechanism to organize content on the page, making it easier to apply styles or layout rules using CSS. For example, a `<div>` can group a heading, a paragraph, and an image together as a single functional unit within a webpage as shown below.

<code>app/Labs/Lab1/page.tsx</code>	How the browser renders
<pre>export default function Lab1() { return (<div id="wd-lab1"> <h2>Lab 1</h2> <h3>HTML Examples</h3> <div id="wd-h-tag"> <h4>Heading Tags</h4> Text documents are often broken up into several sections and subsections. Each section is usually prefaced with a short title or heading that attempts to summarize the topic of the section it precedes. For instance this paragraph is preceded by the heading Heading Tags. The font of the section headings are usually larger and bolder than their subsection headings. This document uses headings to introduce topics such as HTML Documents, HTML Tags, Heading Tags, etc. HTML heading tags can be used to format plain text so that it renders in a browser as large headings. There are 6 heading tags for different sizes: h1, h2, h3, h4, h5, and h6. Tag h1 is the largest heading and h6 is the smallest heading. </div> { /* do the next exercise here */ } </div>); }</pre>	Labs Lab 1 HTML Examples Heading Tags Text documents are often broken up into several sections and subsections. Each section is usually prefaced with a short title or heading that attempts to summarize the topic of the section it precedes. For instance this paragraph is preceded by the heading Heading Tags. The font of the section headings are usually larger and bolder than the plain text and their subsection headings. This document uses headings to introduce topics such as HTML Documents, HTML Tags, Heading Tags, etc. HTML heading tags can be used to format plain text so that it renders in a browser as large headings. There are 6 heading tags for different sizes: h1, h2, h3, h4, h5, and h6. Tag h1 is the largest heading and h6 is the smallest heading.

1.3.2 Formatting Vertical Spacing with the HTML Paragraph Tag

Browsers ignore white spaces such as extra spaces, tabs, and newlines in the source code. To add space between different text blocks, the paragraph tag `<p>` is used to wrap text and add vertical spacing as shown below.

<code>app/Labs/Lab1/page.tsx</code>
<pre><div id="wd-p-tag"> <h4>Paragraph Tag</h4></pre>

```
<p id="wd-p-1">
This is a paragraph. We often separate a long set of sentences with vertical spaces to make the text easier to
read. Browsers ignore vertical white spaces and render all the text as one single set of sentences. To force the
browser to add vertical spacing, wrap the paragraphs you want to separate with the paragraph tag </p>
  /* continue here */
</div>
```

Without these tags, the browser treats text as a single contiguous stream that flows from left to right and wraps only when horizontal space is exhausted. This rendering behavior is referred to as inline layout behavior or just inline. Inline content flows from left to right horizontally the whole width of its parent container and then wraps vertically when there's no more horizontal space. The content below on the left renders as shown below on the right. Notice that even though there are line breaks between the paragraphs on the left, these are ignored by the browser and all text blends together.

app/Labs/Lab1/page.tsx

How the browser renders

```
export default function Lab1() {
  return (
    <div id="wd-lab1">
      <h2>Lab 1</h2>
      <h3>HTML Examples</h3>
      <div id="wd-h-tag"> ... </div>
      <div id="wd-p-tag">
        <h4>Paragraph Tag</h4>
        <p id="wd-p-1"> ... </p>
        This is the first paragraph. The paragraph tag is used to format
        vertical gaps between long pieces of text like this one.

        This is the second paragraph. Even though there is a deliberate white
        gap between the paragraph above and this paragraph, by default browsers
        render them as one contiguous piece of text as shown here on the right.

        This is the third paragraph. Wrap each paragraph with the paragraph tag
        to tell browsers to render the gaps.
      </div>
    </div>
  );
}
```

This is the first paragraph. The paragraph tag is used to format vertical gaps between long pieces of text like this one. This is the second paragraph. Even though we added a deliberate gap between the paragraph above and this paragraph, by default browsers render them as one contiguous piece of text as shown here on the right. This is the third paragraph. Wrap each paragraph with the paragraph tag to tell browsers to render the gaps.

To add the intended vertical spacing on the left, paragraphs can wrap the text to explicitly format margins above and below the text as shown below. Paragraph and heading tags are known as **block elements**. Block elements inherently add vertical space before and after their content and occupy the full width of their parent container. By default, the parent container is the browser window, but elements can be nested and each can be configured independently. By controlling the combination of **inline** and **block** layouts, developers can achieve complex and useful document structures.

app/Labs/Lab1/page.tsx

How the browser renders

```
<div id="wd-p-tag">
  <h4>Paragraph Tag</h4>
  <p id="wd-p-1"> ... </p>
  <p id="wd-p-2">
    This is the first paragraph. The paragraph tag is used to format
    vertical gaps between long pieces of text like this one.
  </p>
  <p id="wd-p-3">
    This is the second paragraph. Even though there is a deliberate white
    gap between the paragraph above and this paragraph, by default
    browsers render them as one contiguous piece of text as shown here on
    the right.
  </p>
  <p id="wd-p-4">
    This is the third paragraph. Wrap each paragraph with the paragraph
    tag to tell browsers to render the gaps.
  </p>
</div>
```

This is the first paragraph. The paragraph tag is used to format vertical gaps between long pieces of text like this one.

This is the second paragraph. Even though there is a deliberate white gap between the paragraph above and this paragraph, by default browsers render them as one contiguous piece of text as shown here on the right.

This is the third paragraph. Wrap each paragraph with the paragraph tag to tell browsers to render the gaps.

```
</p>
</div>
```

1.3.3 Listing Content with the HTML Ordered List Tag

List elements are used to group related items. There are two primary types of lists: **ordered** and **unordered**. Ordered list elements are utilized for sequences where the specific order of items is important, such as procedural steps. The code below demonstrates that even though the steps to make pancakes are enumerated in separate lines, the browser ignores the format and blends the lines into a single paragraph.

app/Labs/Lab1/page.tsx	How the browser renders
<pre>export default function Lab1() { return (<div id="wd-lab1"> <h2>Lab 1</h2> <h3>HTML Examples</h3> <div id="wd-h-tag"> ... </div> <div id="wd-p-tag"> ... </div> <div id="wd-lists"> <h4>List Tags</h4> <h5>Ordered List Tag</h5> How to make pancakes: 1. Mix dry ingredients. 2. Add wet ingredients. 3. Stir to combine. 4. Heat a skillet or griddle. 5. Pour batter onto the skillet. 6. Cook until bubbly on top. 7. Flip and cook the other side. 8. Serve and enjoy! </div> </div>);}</pre>	Paragraph Tag This is a paragraph. We often separate a long set of sentences with vertical spaces to make the text easier to read. Browsers ignore vertical white spaces and render all the text as one single set of sentences. To force the browser to add vertical spacing, wrap the paragraphs you want to separate with the paragraph tag This is the first paragraph. The paragraph tag is used to format vertical gaps between long pieces of text like this one. This is the second paragraph. Even though there is a deliberate white gap between the paragraph above and this paragraph, by default browsers render them as one contiguous piece of text as shown here on the right. This is the third paragraph. Wrap each paragraph with the paragraph tag to tell browsers to render the gaps. List Tags Ordered List Tag How to make pancakes: 1. Mix dry ingredients. 2. Add wet ingredients. 3. Stir to combine. 4. Heat a skillet or griddle. 5. Pour batter onto the skillet. 6. Cook until bubbly on top. 7. Flip and cook the other side. 8. Serve and enjoy!

The desired formatting can be achieved by applying the ordered tag as illustrated below. The ordered list structure consists of a pair of `<ol>` and `</ol>` tags to denote the boundaries of the list, while each item within the list is wrapped in `<li>` and `</li>` tags. The browser automatically handles the numbering, ensuring that the sequence remains intact even if items are added or removed.

app/Labs/Lab1/page.tsx	How the browser renders
<pre><div id="wd-lists"> <h4>List Tags</h4> <h5>Ordered List Tag</h5> How to make pancakes: <ol id="wd-pancakes"> Mix dry ingredients. Add wet ingredients. Stir to combine. Heat a skillet or griddle. Pour batter onto the skillet. Cook until bubbly on top. Flip and cook the other side. Serve and enjoy! </div></pre>	List Tags Ordered List Tag How to make pancakes: 1. Mix dry ingredients. 2. Add wet ingredients. 3. Stir to combine. 4. Heat a skillet or griddle. 5. Pour batter onto the skillet. 6. Cook until bubbly on top. 7. Flip and cook the other side. 8. Serve and enjoy!

Both the `<ol>` and `<li>` elements are **block elements**, forcing vertical separation between items and across the width of the display area. The placeholder below can be used to practice creating ordered lists.

```

<div id="wd-lists">
  <h4>List Tags</h4>
  <h5>Ordered List Tag</h5>
  How to make pancakes:
  <ol id="wd-pancakes"> ... </ol>
  My favorite recipe:
  <ol id="wd-your-favorite-recipe">
    { /* complete on your own */ }
  </ol>
</div>

```

1.3.4 Listing Content In No Particular Order with the HTML Unordered List Tag

Unordered list elements are used to display a collection of items where the specific order does not change the meaning of the content. Instead of numbers, the browser uses bullets to decorate each line item. The unordered list is defined by the `<ul>` tag, while individual items still use the `<li>` tag. Similar to ordered lists, the `<ul>` element is a **block element**. The code below illustrates how to implement an unordered list.

```

My favorite recipe:
<ol id="wd-my-favorite-recipe">{ /* complete on your own */ }</ol>
<h5>Unordered List Tag</h5>
My favorite books (in no particular order)
<ul id="wd-my-books">
  <li>Dune</li>
  <li>Lord of the Rings</li>
  <li>Ender's Game</li>
  <li>Red Mars</li>
  <li>The Forever War</li>
</ul>
Your favorite books (in no particular order)
<ul id="wd-your-books">
  { /* complete on your own */ }
</ul>

```

Unordered List Tag

My favorite books (in no particular order)

- Dune
- Lord of the Rings
- Ender's Game
- Red Mars
- The Forever War

Your favorite books (in no particular order)

1.3.5 Tabulating Data with the HTML Table Tags

HTML began as a tool for formatting and sharing scientific documents and research results among physicists worldwide. These documents frequently included structured experimental data — such as measurements with multiple attributes (e.g., speed, temperature, location) — often presented in tabular form. Although tables were not part of the original HTML, the `<table>` tag was later added in the mid-1990s specifically to allow data to be organized into rows and columns. For example, grades from multiple quizzes taken over a semester could be clearly displayed using a table as shown below.

Quiz	Topic	Date	Grade
Q1	HTML	2/3/21	85
Q2	CSS	2/10/21	90
Q3	JavaScript	2/17/21	95
Average			90

Several things to note:

1. The first row is formatted as headings for each column
2. There are 3 data points, one for each quiz, one in each row
3. Data under the same column is of the same data type

4. The last row is formatted as a footer
5. The three first columns of the last row are merged into a single cell and unlike the 3 data rows. Rows also can span to merge into a single cell.

HTML **table** tag can contain additional tags to format the data as follows:

- **table** - declares the start of a table
- **tr** - declares the start of a row
- **td** - declares a table data cell
- **thead** - declares a row of headings
- **tbody** - declares the main data content rows of the table
- **tfoot** - declares a row as a footer
- **th** - declares a table cell as a heading

Tables allow for complex data organization where multiple attributes of a single entity (like a quiz) are aligned across rows and columns. This semantic grouping ensures that data remains readable and structured for the end user. The following code demonstrates how to implement an HTML table for displaying quiz grades.

app/Labs/Lab1/page.tsx

```
export default function Lab1() {
  return (
    <div id="wd-lab1">
      <h2>Lab 1</h2>
      <h3>HTML Examples</h3>
      ...
      <div id="wd-tables">
        <h4>Table Tag</h4>
        <table border={1} width="100%">
          <thead>
            <tr>
              <th>Quiz</th>
              <th>Topic</th>
              <th>Date</th>
              <th>Grade</th>
            </tr>
          </thead>
          <tbody>
            <tr>
              <td>Q1</td>
              <td>HTML</td>
              <td>2/3/21</td>
              <td>85</td>
            </tr>
            <tr>
              <td>Q2</td>
              <td>CSS</td>
              <td>2/10/21</td>
              <td>90</td>
            </tr>
            <tr>
              <td>...</td>
            </tr>
          </tbody>
          <tfoot>
            <tr>
              <td colspan={3}>Average</td>
              <td>90</td>
            </tr>
          </tfoot>
        </table>
      </div>
    </div>
  );
}
```

<!-- declares the table, sets border and width -->
<!-- declares the table heading section -->
<!-- declares the headings row -->
<!-- declares heading for first column -->
<!-- declares heading for second column -->
<!-- declares heading for third column -->
<!-- declares heading for fourth column -->

<!-- declares the table's main content -->
<!-- declares the first row -->
<!-- declares data for first row, first column -->
<!-- declares data for first row, second column -->
<!-- declares data for first row, third column -->
<!-- declares data for first row, fourth column -->

<!-- declares the second row -->
<!-- declares data for second row, first column -->
<!-- declares data for second row, second column -->
<!-- declares data for second row, third column -->
<!-- declares data for second row, fourth column -->

<!-- declares the third row -->

<!-- declares the table footer section -->
<!-- declares the footer row -->
<!-- declares data spans 3 columns -->
<!-- declares data for fourth column -->

Table content can be further refined using alignment attributes. The **valign** attribute controls vertical alignment (e.g., "top"), while the **align** attribute controls horizontal alignment (e.g., "right") within individual cells.

1.3.6 Image Tag

The image tag is used to render pictures in HTML documents. The images can be anywhere on the internet, or a local image document in the local file system.

```

```

<!-- Use img tag to embed pictures in HTML documents. src attributes references image file either locally or remotely. width and height attributes configure the image size. If only width or height is provided, the other scales proportionally -->

Images can be sourced from remote servers using absolute URLs or from the local file system using relative paths. When using Next.js, local images are typically stored in the **public/images** directory in the current project as shown below.

```
<div id="wd-images">
  <h4>Image tag</h4>
  Loading an image from the internet: <br />
  
  <br />
  Loading a local image:
  <br />
  </div>
```

Image tag

Loading an image from the internet:



1.3.7 Creating Web Forms

Form tags are useful for entering data. Let's take a look at the most common ones: **form**, **input**, **select**, **textarea**, **radio**, **checkbox**.

1.3.7.1 Creating Text Input Fields

Text fields are the most common form elements allowing entering a single line of text.

```
<input type="text"
      placeholder="hint"
      title="tooltip"
      value="COMEDY"/>
```

<!-- use input tag's text type to declare a single line input field text is default if type is left out. Use placeholder and title to give a hint of what information you're expecting. Optionally initialize the value of the field with value attribute-->

The **label** tag is essential for accessibility and usability. By associating a label with an input field through the **htmlFor** attribute, clicking the label text will automatically give focus to or toggle the corresponding input element. The following code snippet demonstrates the implementation of a form containing several text input fields, each associated with a label element.

```
<div id="wd-forms">
  <h4>Form Elements</h4>
  <form id="wd-text-fields">
    <h5>Text Fields</h5>
    <label htmlFor="wd-text-fields-username">Username:</label>
    <input placeholder="jdoe" id="wd-text-fields-username" /> <br />
    <label htmlFor="wd-text-fields-password">Password:</label>
    <input type="password" defaultValue="123@#$asd"
id="wd-text-fields-password" /><br />
    <label htmlFor="wd-text-fields-first-name">First name:</label>
    <input type="text" title="John" id="wd-text-fields-first-name" /> <br />
    <label htmlFor="wd-text-fields-last-name">Last name:</label>
    <input type="text" placeholder="Doe"
defaultValue="Wonderland"
title="The last name" />
  </form>
</div>
```

Form Elements

Text Fields

Username:

Password:

First name:

Last name:

```

        id="wd-text-fields-last-name" />
    { /* copy rest of form elements here */ }
</form>
</div>

```

1.3.7.2 Creating Multi-Line Input Fields with Textarea

The **textarea** tag is useful for entering long form text such as someone's biography data, or a blog post.

```

<textarea cols="20" rows="25"
    placeholder="Biography"
    title="tooltip">Some text
</textarea>

```

<!-- use textarea tag for long form text configure its width and height with attributes cols and rows. Use placeholder and tooltip to give hints. Note default value is in tag's body -->

Unlike single-line text inputs, the **textarea** element allows for large volumes of text. It is a container tag, meaning its default value is placed within the body of the tag rather than as a value attribute.

```

<h5>Text boxes</h5>
<label>Biography:</label><br/>
<textarea id="wd-textarea" cols={30} rows={10}>Lorem ipsum dolor sit
amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis
nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate velit
esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat
cupidatat non proident, sunt in culpa qui officia deserunt mollit
anim id est laborum.</textarea>

```

Text boxes

Biography:

Lorem ipsum dolor sit amet,
 consectetur adipiscing elit,
 sed do eiusmod tempor
 incididunt ut labore et dolore
 magna aliqua. Ut enim ad minim
 veniam, quis nostrud
 exercitation ullamco laboris
 nisi ut aliquip ex ea commodo
 consequat. Duis aute irure
 dolor in reprehenderit in

1.3.7.3 Creating Single Choices with HTML Radio Buttons

Radio buttons allow selecting a single choice from multiple alternative options. Radio buttons are grouped to create a "single-choice" selection. By assigning the same **name** attribute to multiple radio inputs, the browser ensures that only one option in that group can be selected at a time.

```

<input type="radio"
    name="NAME1"
    value="OPTION1"/>
<input type="radio"
    name="NAME1"
    value="OPTION2" checked/>

```

/ use the input tag's radio type to declare radio buttons */*
/ group radio buttons by using the same name across several buttons */*
/ use checked to set the checkbox's as initially checked */*

The example below illustrates four radio buttons from which only one option can be selected. The labels associated with the buttons allow clicking the label text to select the associated radio button, improving the overall user experience.

```

<h5 id="wd-radio-buttons">Radio buttons</h5>

<label>Favorite movie genre:</label><br />

<input type="radio" name="radio-genre" id="wd-radio-comedy"/>
<label htmlFor="wd-radio-comedy">Comedy</label><br />

<input type="radio" name="radio-genre" id="wd-radio-drama"/>
<label htmlFor="wd-radio-drama">Drama</label><br />

<input type="radio" name="radio-genre" id="wd-radio-scifi"/>
<label htmlFor="wd-radio-scifi">Science Fiction</label><br />

<input type="radio" name="radio-genre" id="wd-radio-fantasy"/>
<label htmlFor="wd-radio-fantasy">Fantasy</label>

```

Radio buttons

Favorite movie genre:

- ☐ Comedy
- ☐ Drama
- ☒ Science Fiction
- ☐ Fantasy

1.3.7.4 Creating Multiple Selections with HTML Checkboxes

Checkboxes allow selecting multiple choices. Assigning the same name attribute to a group of checkboxes allows the browser to associate these fields as a set, which is useful for grouping multiple selected values together when the form is submitted.

```
<input type="checkbox" name="NAME2" value="OPTION1" checked/>
<input type="checkbox" name="NAME2" value="OPTION2"/>
<input type="checkbox" name="NAME2" value="OPTION3" checked/>
```

/ use the input tag's checkbox type to declare a checkbox */*
/ use the same name value to group multiple checkboxes */*
/ use checked attribute to select the checkbox by default */*

Checkboxes are used when the user is permitted to select multiple independent options. Unlike radio buttons, selecting one checkbox does not automatically deselect others in the same group. The example below demonstrates four checkboxes from which any number of options can be selected. The labels associated with the boxes allow clicking the label text to toggle the state of the associated checkbox, significantly improving the overall user experience and accessibility.

```
<h5 id="wd-checkboxes">Checkboxes</h5>
<label>Favorite movie genre:</label><br/>
<input type="checkbox" name="check-genre" id="wd-chkbox-comedy"/>
<label htmlFor="wd-chkbox-comedy">Comedy</label><br/>
<input type="checkbox" name="check-genre" id="wd-chkbox-drama"/>
<label htmlFor="wd-chkbox-drama">Drama</label><br/>
<input type="checkbox" name="check-genre" id="wd-chkbox-scifi"/>
<label htmlFor="wd-chkbox-scifi">Science Fiction</label><br/>
<input type="checkbox" name="check-genre" id="wd-chkbox-fantasy"/>
<label htmlFor="wd-chkbox-fantasy">Fantasy</label>
```

Checkboxes

Favorite movie genre:

- ☒ Comedy
☐ Drama
☒ Science Fiction
☐ Fantasy

1.3.7.5 Creating Dropdown Menus with HTML Select and Option

Dropdowns are useful for selecting one or more options from a list of possible values. The default version displays a set of values from which one can choose a single value.

```
<select>
  <option value="VAL1">Value 1</option>
  <option value="VAL2" selected>Value 2</option>
  <option value="VAL3">Value 3</option>
</select>
```

*<!-- Wrap several option tags in a select tag.
Optionally provide options value, otherwise the option's
text is the value of the select element. Optionally use
selected attribute to select default. -->*

Adding the optional **multiple** attribute converts the dropdown into a list of options that can be selected.

```
<select multiple>
  <option value="VAL1" selected>Value 1</option>
  <option value="VAL2">Value 2</option>
  <option value="VAL3" selected>Value 3</option>
</select>
```

*<!-- Alternatively use attribute multiple to allow
selecting more than one option. Use ctrl+click to select
more than one option -->*

The select element provides a compact way to present many options. The browser typically renders this as a clickable dropdown menu. Using the **multiple** attribute changes the interaction to allow multiple selections via keyboard modifiers like **Shift**, **Ctrl** or **Command**. The first **<select>** element in the code below creates a single-select dropdown menu, where users can choose exactly one option from the list. The second **<select>** element includes the **multiple** attribute, transforming it into a multi-select list that allows users to select several

options by using keyboard modifiers like Shift, Ctrl, or Command. Within both, the **<option>** tags represent the individual choices available to the user. Each **<option>** includes a **value** attribute, which defines the specific data sent to the server when the form is submitted, while the text content between the option tags serves as the label displayed to the user in the menu.

```
<h4 id="wd-dropdowns">Dropdowns</h4>

<h5>Select one</h5>
<label htmlFor="wd-select-one-genre"> Favorite movie genre: </label><br/>
<select id="wd-select-one-genre">
  <option value="COMEDY">Comedy</option>
  <option value="DRAMA">Drama</option>
  <option selected value="SCIFI">
    Science Fiction</option>
  <option value="FANTASY">Fantasy</option>
</select>

<h5>Select many</h5>
<label htmlFor="wd-select-many-genre"> Favorite movie genres:
</label><br/>
<select multiple id="wd-select-many-genre">
  <option value="COMEDY" selected> Comedy </option>
  <option value="DRAMA"> Drama </option>
  <option value="SCIFI" selected> Science Fiction </option>
  <option value="FANTASY"> Fantasy </option>
</select>
```

Dropdowns

Select one

Favorite movie genre:

Science Fiction ▼

Select many

Favorite movie genres:

Comedy
Drama
Science Fiction
Fantasy

1.3.7.6 Other HTML Field Types

The input tag's **type** attribute has several other possible values: **date**, **email**, **number**, and **range** which configure the **input** tag to handle different data formats. The code below demonstrates several

```
<h4>Other HTML field types</h4>

<label htmlFor="wd-text-fields-email"> Email: </label>
<input type="email"
  placeholder="jdoe@somewhere.com"
  id="wd-text-fields-email"/><br/>

<label htmlFor="wd-text-fields-salary-start"> Starting salary:</label>
<input type="number"
  defaultValue="100000"
  placeholder="1000"
  id="wd-text-fields-salary-start"/><br/>

<label htmlFor="wd-text-fields-rating"> Rating: </label>
<input type="range"
  defaultValue="4"
  max="5"
  placeholder="Doe"
  id="wd-text-fields-rating"/><br/>

<label htmlFor="wd-text-fields-dob"> Date of birth: </label>
<input type="date"
  defaultValue="2000-01-21"
  id="wd-text-fields-dob"/><br/>
```

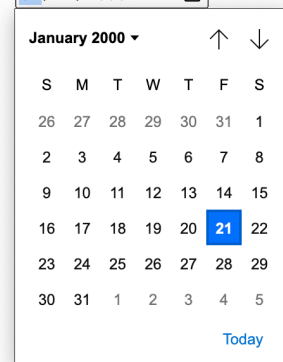
Other HTML field types

Email: jannunzi@gmail.com

Starting salary: 100000

Rating:

Date of birth: 01/21/2000



1.3.8 Implementing Navigation with the HTML Anchor Tag

The anchor tag renders text encouraging users to click on it to navigate to other websites or pages.

```
<a href="aa.com">
American Airlines</a>

<!-- Use the href attribute to refer to the location of the website or other
page in the same website. Click on the body text to navigate -->
```

The anchor tag `<a>` is the fundamental building block of web navigation. It uses the **href** attribute to define the destination URL, allowing users to jump to external websites or other documents.

```
<h4>Anchor tag</h4>
Please
<a href="https://www.lipsum.com" id="wd-lipsum">click here</a>
to get dummy text<br/>
```

Anchor tag

Please [click here](#) to get dummy text

1.3.9 Implementing Navigation

Implementing navigation in **Single Page Applications (SPAs)** is different than with anchor tags since in SPAs don't navigate away from the current webpage, but instead show or hide screens and components based on the URL. For instance, there might be screens **Signin** and **Profile** mapped to URL **paths**, or **routes** **/signin** and **/profile**. When the **URL** contains the corresponding path, then SPA applications will display the corresponding content.

As you work through the various chapters, we're going to work on various exercises discussing different topics. To practice some more navigation, create placeholders for upcoming lab exercises and implement navigation to each of the labs. Create the following **Lab2** and **Lab3** components as shown below.

app/Labs/Lab2/page.tsx

```
export default function Lab2() {
  return (
    <div>
      <h2>Lab 2</h2>
    </div>
  );}
```

app/Labs/Lab3/page.tsx

```
export default function Lab3() {
  return (
    <div>
      <h2>Lab 3</h2>
    </div>
  );}
```

While standard anchor tags work, Next.js provides a specialized **Link** component. This component is optimized for internal navigation within a Single Page Application, enabling faster page transitions without a full browser reload.

app/Labs/page.tsx

<http://localhost:3000/Labs>

```
import Link from "next/link";
export default function labs() {
  return (
    <div id="wd-labs">
      <h1>Labs</h1>
      <ul>
        <li>
          <Link href="/labs/lab1" id="wd-lab1-link">
            Lab 1: HTML Examples </Link>
          </li>
        <li>
          <Link href="/labs/lab2" id="wd-lab2-link">
            Lab 2: CSS Basics </Link>
          </li>
        <li>
          <Link href="/labs/lab3" id="wd-lab3-link">
            Lab 3: JavaScript Fundamentals </Link>
          </li>
        </ul>
      </div>
    );}
```

Labs

- [Lab 1: HTML Examples](#)
- [Lab 2: CSS Basics](#)
- [Lab 3: JavaScript Fundamentals](#)

1.3.10 Implementing Layouts

In Next.js, layouts are a powerful feature that allow developers to create consistent and reusable UI structures across multiple pages or routes. By utilizing the **layout.tsx** file within the app directory, you can define a shared

layout that wraps specific pages or child routes, ensuring a uniform appearance and functionality. NOTE: make sure the name of the **layout.tsx** file must be all lower caps. The code below demonstrates a **LabsLayout** component that integrates a **table of contents (TOC)** as a sidebar navigation, alongside the main content rendered via the **children** prop. The TOC component, defined in **TOC.tsx**, generates a list of navigational links, which are embedded into the layout to create a cohesive user experience across the "Labs" section of the application.

app/Labs/TOC.tsx	app/Labs/Layout.tsx
<pre>import Link from "next/link"; export default function TOC() { return (<Link href="/labs" id="wd-home-link"> Home </Link> <Link href="/labs/lab1" id="wd-lab1-link"> Lab 1 </Link> <Link href="/labs/lab2" id="wd-lab2-link"> Lab 2 </Link> <Link href="/labs/lab3" id="wd-lab3-link"> Lab 3 </Link>); }</pre>	<pre>import { ReactNode } from "react"; import TOC from "../TOC"; export default function LabsLayout({ children, }: Readonly<{ children: ReactNode }>) { return (<table> <tbody> <tr> <td valign="top" width="100px"> <TOC /> </td> <td valign="top">{children}</td> </tr> </tbody> </table>); }</pre>

The **children** prop in the **LabsLayout** component is dynamically populated with the content of the **page.tsx** file located in the **app/labs** directory. In this case, the **page.tsx** file defines the **Labs** component, which renders a main page for the "Labs" section containing a list of links to individual the lab page. When the **/labs** route is accessed, the **LabsLayout** wraps this content, placing the **TOC** component in a sidebar column and the **Labs** component's content in the main column, ensuring a consistent layout across all pages under the **/labs** route. Confirm that navigating to <http://localhost:3000/labs> shows the content below.

- [Home](#)
- [Lab 1](#)
- [Lab 2](#)
- [Lab 3](#)

Labs

- [Lab 1: HTML Examples](#)
- [Lab 2: CSS Basics](#)
- [Lab 3: JavaScript Fundamentals](#)

Figure 1.3.10 - Labs with layout

1.3.8 Exercises

To practice the concepts introduced in the preceding sections, implement the following tasks in your **app/labs/lab1/page.tsx** file:

1. Implement the heading and div examples shown in section 1.3.1.
2. Apply paragraph tags to create vertical gaps between the text blocks provided in section 1.3.2.

3. Create an ordered list for a pancake recipe and another for your own favorite recipe with at least 3 steps (Section 1.3.3).
4. Implement an unordered list of my favorite books and add a new list for 3 of your own favorite books (Section 1.3.4).
5. Build the quiz data table and add the 7 missing quizzes (Q3-Q10) with custom dates and scores (Section 1.3.5).
6. Embed the remote Starship image and a local "teslabot.jpg" image from your public directory (Section 1.3.6).
7. Create the personal information form including text fields, textarea, radio buttons, checkboxes, and dropdown menus (Section 1.3.7).
8. Implement the anchor tag example and add a link to your GitHub repository with the ID "wd-github" (Section 1.3.8).
9. Set up the navigation links to Lab 1, Lab 2, and Lab 3 using the Next.js Link component (Section 1.3.9).

1.4 Prototyping the React Kambaz User Interface with HTML

So far the exercises in this chapter have served to practice various aspects of **HTML** and gain confidence with building user interfaces. The following sections will put those skills to use by building **Kambaz**, a Web site based on a popular **Online Learning Management System (OLMS)**. This chapter will focus on implementing some simple, prototype versions of the more common screens and later chapters will incrementally improve on the prototype. Create a new directory **app/kambaz** and do all work in there.

1.4.1 Implementing the Kambaz Landing Page

A landing page represents the primary entry point and default screen of a web application. It serves as the initial destination when a user navigates to the root URL. In Next.js, the routing system automatically maps the file structure within the **app** directory to URL paths. By default, a **page.tsx** file at the root of the **app** folder serves as the application's homepage. However, to maintain a clean and modular architecture, Next.js utilizes **route groups**. By naming a directory with parentheses, such as **(kambaz)**, developers can organize related routes and layouts without affecting the URL path. This allows the Kambaz application logic to reside in its own dedicated directory while still being treated as root-level content, ensuring that the primary application experience is seamlessly delivered at the main domain address.

1.4.1.1 Exercises

1. Create the Kambaz landing page component in **app/(kambaz)/page.tsx** using the source code provided in the example below.

app/(kambaz)/page.tsx

```
export default function Kambaz() {
  return (
    <div id="wd-kambaz">
      <h1>Kambaz</h1>
    </div>
  );}
```

2. To make Kambaz the default application, remove the existing root **app/page.tsx** and ensure your project directory is named **(kambaz)** with parentheses.
3. Update **TOC.tsx** and the Labs page to include navigation links to the new root-level Kambaz application.

- Verify that navigating to the root URL (e.g., <http://localhost:3000>) correctly displays the Kambaz landing page.

`app/Labs/TOC.tsx`

```
import Link from "next/link";
export default function TOC() {
  return (
    <ul>
      <li>
        <Link href="/labs" id="wd-lab1-link">
          Home </Link> </li>
        ...
      <li>
        <Link href="/" id="wd-kambaz-link">
          Kambaz </Link> </li>
      </ul>
    );
  }
```

<http://localhost:3000/Labs>

- [Home](#)
- [Lab 1](#)
- [Lab 2](#)
- [Lab 3](#)
- [Kambaz](#)

Labs

- [Lab 1: HTML Examples](#)
- [Lab 2: CSS Basics](#)
- [Lab 3: JavaScript Fundamentals](#)
- [Kambaz](#)

1.4.2 The Kambaz Account Screens

The **Account** screens provide access to personal information, such as registered courses and teaching assignments. The **Sign Up** screen manages application registration, while the **Sign In** screen handles user identification. The **Profile** screen enables the viewing and modification of personal details. Figures 1.4.2a through 1.4.2d illustrate the visual goals for these screens.

Sign in

[Sign up](#)

Figure 1.4.2a
The Sign in screen

Sign up

[Sign in](#)

Figure 1.4.2b
The Sign up screen

Profile

Figure 1.4.2c
The Profile screen

[Signin](#) [Signup](#) Sign in [Profile](#)

[Sign in](#)

[Sign up](#)

Figure 1.4.2d
Account Navigation

1.4.2.1 The Sign In Screen

The **Sign In** screen grants application access upon the submission of valid credentials. The **Signin** component defines the input fields necessary for user identification. The **Link** component in the **next/link** module facilitates navigation between authentication screens, while the **redirect** function from **next/navigation** enforces the default landing page.

`app/(kambaz)/account/signin/page.tsx`

```
import Link from "next/link";
export default function Signin() {
  return (
    <div id="wd-signin-screen">
```

```

    <h3>Sign in</h3>
    <input placeholder="username" className="wd-username" defaultValue="ada" /> <br />
    <input placeholder="password" type="password" className="wd-password" defaultValue="123" /> <br />
    <Link href="/profile" id="wd-signin-btn"> Sign in </Link> <br />
    <Link href="/signup" id="wd-signup-link"> Sign up </Link>
  </div>
);}

```

Create an **Account** screen that redirects to the **Signin** screen so that it's the default screen. Confirm that navigating to <http://localhost:3000/account> does indeed redirect to <http://localhost:3000/account/signin>.

app/(kambaz)/account/page.tsx

```

import { redirect } from "next/navigation";

export default function AccountPage() {
  redirect("/account/signin");
}

```

In the **Kambaz** page, redirect to the **Signin** screen so that it's also the default screen. Confirm that navigating to <http://localhost:3000> does indeed redirect to <http://localhost:3000/account/signin>.

app/(kambaz)/page.tsx

```

import { redirect } from "next/navigation";
export default function Kambaz() {
  redirect("/account/signin");
  return (
    <div id="wd-kambaz">
      <h1>Kambaz</h1>
    </div>
  );
}

```

1.4.2.2 Implementing the Kambaz Sign Up Screen HTML Layout

The **Sign Up** screen expands registration functionality by requiring a password verification field. The **Signup** component below mirrors the structure of the **Sign In** screen, introducing the additional input field necessary for account creation. This layout ensures a consistent user experience during the onboarding process.

app/(kambaz)/account/signup/page.tsx

```

import Link from "next/link";
export default function Signup() {
  return (
    <div id="wd-signup-screen">
      <h3>Sign up</h3>
      <input placeholder="username" className="wd-username" defaultValue="ada" /><br/>
      <input placeholder="password" type="password" className="wd-password" defaultValue="123" /><br/>
      <input placeholder="verify password"
        type="password" className="wd-password-verify" /><br/>
      <Link href="/profile" > Sign up </Link><br />
      <Link href="/signin" > Sign in </Link>
    </div>
  );
}

```

1.4.2.3 Implementing the Kambaz Profile Screen HTML Layout

The **Profile** screen displays a comprehensive view of user data. The **Profile** component below utilizes HTML input types to process various data formats, including **date** for birth dates and **email** for contact information. The **select** element provides a constrained set of options for role management, as shown in the following implementation.

app/(kambaz)/account/profile/page.tsx

```
import Link from "next/link";
export default function Profile() {
  return (
    <div id="wd-profile-screen">
      <h3>Profile</h3>
      <input defaultValue="alice" placeholder="username" className="wd-username" /><br/>
      <input defaultValue="123" placeholder="password" type="password"
        className="wd-password" /><br/>
      <input defaultValue="Alice" placeholder="First Name" id="wd-firstname" /><br/>
      <input defaultValue="Wonderland" placeholder="Last Name" id="wd-lastname" /><br/>
      <input defaultValue="2000-01-01" type="date" id="wd-dob" /><br/>
      <input defaultValue="alice@wonderland" type="email" id="wd-email" /><br/>
      <select defaultValue="FACULTY" id="wd-role">
        <option value="USER">User</option> <option value="ADMIN">Admin</option>
        <option value="FACULTY">Faculty</option> <option value="STUDENT">Student</option>
      </select><br/>
      <Link href="signin" > Sign out </Link>
    </div>
  );
}
```

1.4.2.4 Implementing Account Navigation

The **Account Navigation** sidebar enables transitions between account-related screens without reloading the application. The **AccountNavigation** component below defines the link structure, while the **AccountLayout** component integrates this sidebar into a two-column table structure, ensuring consistent navigation across the account module.

app/(kambaz)/account/Navigation.tsx

```
import Link from "next/link";
export default function AccountNavigation() {
  return (
    <div id="wd-account-navigation">
      <Link href="signin"> Signin </Link> <br />
      <Link href="signup"> Signup </Link> <br />
      <Link href="profile"> Profile </Link> <br />
    </div>
  );
}
```

In the **Account** screen add a **table** to layout the **AccountNavigation** component on a column on the left and the routes on the right side column as shown below. Confirm that the **Sign in** screen is the default screen as shown in Figure 1.4.2d. Also confirm that clicking the links in the **Account Navigation** sidebar actually navigates to the screens

app/(kambaz)/account/Layout.tsx

```
import { ReactNode } from "react";
import AccountNavigation from "../Navigation";
export default function AccountLayout({ children }: Readonly<{ children: ReactNode }>) {
  return (
    <div id="wd-kambaz-account">
      <table>
        <tbody>
          <tr>
            <td valign="top"> <AccountNavigation /> </td>
            <td valign="top" width="100%"> {children} </td>
          </tr>
        </tbody>
      </table>
    </div>
  );
}
```

1.4.2.5 Exercises

5. Create the Signin component in `app/(kambaz)/account/signin/page.tsx` as described in section 1.4.2.1.
6. Implement a redirect from `app/(kambaz)/account/page.tsx` and `app/(kambaz)/page.tsx` to the Signin screen.
7. Implement the Signup screen in `app/(kambaz)/account/signup/page.tsx` following the layout in section 1.4.2.2.
8. Create the Profile screen in `app/(kambaz)/account/profile/page.tsx` including role selection as detailed in section 1.4.2.3.
9. Develop the Account Navigation component in `app/(kambaz)/account/Navigation.tsx` and integrate it into a two-column table within `app/(kambaz)/account/layout.tsx` (Section 1.4.2.4).
10. Update `app/(kambaz)/account/layout.tsx` to use a two-column table layout with the navigation sidebar on the left and page content on the right.
11. Create the **Account Navigation** component in `app/(kambaz)/account/Navigation.tsx` providing links to Signin, Signup, and Profile. Verify navigation flows between Signin, Signup, and Profile via the sidebar.

1.4.3 Implementing the Kambaz Dashboard Screen

The **Kambaz Dashboard** lists courses as illustrated below, giving access to courses a user is enrolled in as a student and courses a faculty is teaching.

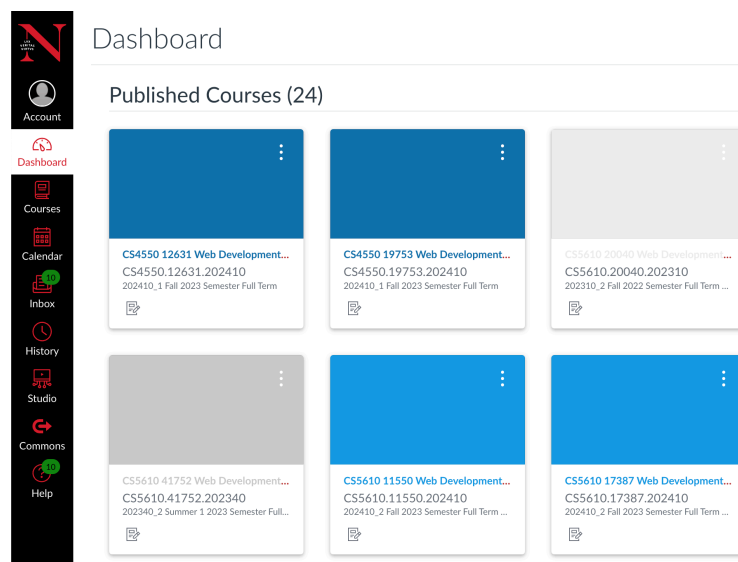


Figure 1.4.3 - The Dashboard Screen

Clicking on a course navigates to that specific course. The **Kambaz Dashboard** displays multiple courses, and each course item contains a link to its unique route. This screen utilizes Next.js Link and Image components to render a responsive grid of available learning materials. Refer to the code below for the Dashboard page implementation. Feel free to lookup for images on the internet or generate images with your favorite AI tool.

`app/(kambaz)/dashboard/page.tsx`

```
import Link from "next/link";
import Image from "next/image";
export default function Dashboard() {
  return (
```

```

<div id="wd-dashboard">
  <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
  <h2 id="wd-dashboard-published">Published Courses (12)</h2> <hr />
  <div id="wd-dashboard-courses">
    <div className="wd-dashboard-course">
      <Link href="/courses/1234" className="wd-dashboard-course-link">
        <Image src="/images/reactjs.jpg" width={200} height={150} alt="reactjs" />
        <div>
          <h5> CS1234 React JS </h5>
          <p className="wd-dashboard-course-title">
            Full Stack software developer
          </p>
          <button> Go </button>
        </div>
      </Link>
    </div>
    <div className="wd-dashboard-course"> ... </div>
    <div className="wd-dashboard-course"> ... </div>
  </div>
</div>
);}

```

{ /* Dashboard Title */}
 { /* Published Courses */}
 { /* Course 1 */}

 { /* Course 2 */}
 { /* Course 3 */}
 { /* Add at Least 7
 courses in total */}

The dashboard implemented in this chapter does not have to look exactly like the screen shot shown above. Later chapters will revisit this screen and apply **CSS** to style it as shown in the screen shot. Change the **Sign in** link in the **Sign In** screen so that it navigates to the new **Dashboard** screen. Confirm that signing in navigates to the **Dashboard**. Confirm you can still navigate to the **Account** screens with the **Account Navigation Sidebar**.

app/(kambaz)/account/signin/page.tsx

```

...
<div id="wd-signin-screen">
  <h3>Sign in</h3>
  <input className="wd-username" placeholder="username" defaultValue="ada" /> <br />
  <input className="wd-password" placeholder="password" type="password" defaultValue="123" /> <br />
  <Link id="wd-signin-btn" href="/dashboard"> Sign in </Link> <br />
  <Link id="wd-signup-link" href="signup"> Sign up </Link>
</div>

```

1.4.3.1 Implementing the Kambaz Navigation Sidebar

The **Kambaz Navigation** sidebar acts as the primary global navigation for the entire Kambaz application. It consists of a list of hyperlinks that allow users to jump between the Dashboard, Account, Calendar, and Inbox. The sidebar is placed within the main application layout to ensure it remains visible as users interact with different screens.

app/(kambaz)/Navigation.tsx

```

import Link from "next/link";
export default function KambazNavigation() {
  return (
    <div id="wd-kambaz-navigation">
      <a href="https://www.northeastern.edu/" id="wd-neu-link" target="_blank">
        Northeastern</a><br/>
      <Link href="/account" id="wd-account-link">Account</Link><br/>
      <Link href="/dashboard" id="wd-dashboard-link">Dashboard</Link><br/>
      <Link href="/dashboard" id="wd-course-link">Courses</Link><br/>
      <Link href="/calendar" id="wd-calendar-link">Calendar</Link><br/>
      <Link href="/inbox" id="wd-inbox-link">Inbox</Link><br/>
      <Link href="/labs" id="wd-labs-link">Labs</Link><br/>
    </div>
  );}

```

[Northeastern](#)
[Account](#)
[Dashboard](#)
[Courses](#)
[Calendar](#)
[Inbox](#)
[Labs](#)

In a **Kambaz** layout, use the **table** element to layout the **Kambaz Navigation** and the **Dashboard** into a single row with two columns. Put the **Kambaz Navigation** sidebar on the left column, and the **children** on the right. Confirm that the **Kambaz** application renders as shown below.

app/(kambaz)/Layout.tsx

```
import { ReactNode } from "react";
import KambazNavigation from "../Navigation";
export default function KambazLayout({ children }: Readonly<{ children: ReactNode }>) {
  return (
    <table>
      <tbody>
        <tr>
          <td valign="top" width="200"> <KambazNavigation /> </td>
          <td valign="top" width="100%"> {children} </td>
        </tr>
      </tbody>
    </table>
  );
}
```

1.4.3.2 Handling Missing Pages

If users click on **Calendar** or **Inbox** they'll get a 404 since the pages don't exist. We can handle missing content gracefully by creating a **not-found.tsx** page at the root of the application as shown below. Confirm that navigating to **Calendar** or **Inbox** does not cause a 404 error and instead allows users to navigate back to the **Dashboard**.

app/not-found.tsx

```
import Link from "next/link";
export default function NotFound() {
  return (
    <div className="p-4">
      <h2 className="text-danger">Page Not Found</h2>
      <p>
        The requested page could not be found. Please check the page URL or
        return to the dashboard.
      </p>
      <Link href="/dashboard" className="btn btn-primary mt-3">
        Back to Dashboard
      </Link>
    </div>
  );
}
```

1.4.3.3 Exercises

12. Implement the **Kambaz Dashboard** in *app/(kambaz)/dashboard/page.tsx* displaying at least 7 courses with custom images and links.
13. Create the **Kambaz Navigation** sidebar in *app/(kambaz)/Navigation.tsx* including links for Northeastern, Account, Dashboard, Calendar, and Inbox.
14. Update *app/(kambaz)/layout.tsx* to use a table to layout the Kambaz Navigation on the left and Dashboard content on the right.
15. Update the **Sign in** link in *app/(kambaz)/account/signin/page.tsx* to navigate to the new **/dashboard** screen.
16. Implement *app/not-found.tsx* to handle missing pages like Calendar or Inbox gracefully.

1.4.4 Implementing the Courses Screen

Clicking a course in the **Dashboard** navigates to the **Courses** screen displaying a course's content as shown below.

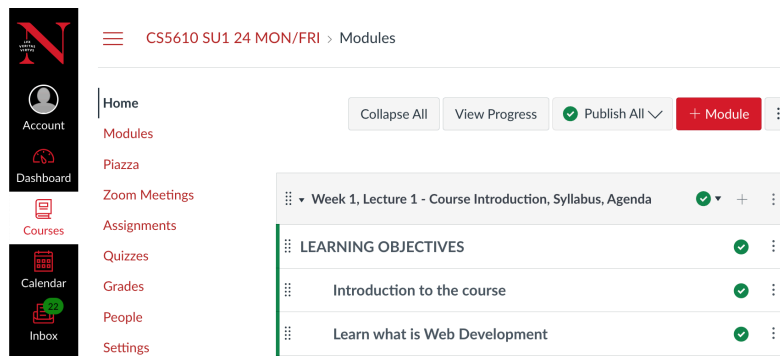


Figure 1.4.4 - The Courses Screen

Create a placeholder for the **Course** screen in a new file `app/(kambaz)/courses/[cid]/page.tsx`.

`app/(kambaz)/courses/[cid]/page.tsx`

```
export default function Courses() {
  return (
    <div id="wd-courses">
      <h2>Course 1234</h2>
    </div>
  );
}
```

1.4.4.1 Implementing the Kambaz Course Navigation Sidebar

When navigating to the **Courses** screen, a second column displays a **Course Navigation** sidebar to navigate to various screens related to that course. In this chapter we'll implement the **Home**, **Module**, and **Assignments** links that navigate to corresponding screens. For the rest of the links, implement placeholder pages that just display a simple heading, e.g., **Piazza**, **Zoom**, **Quizzes**, etc. In `app/(kambaz)/courses/[cid]/Navigation.tsx` create the **CourseNavigation** sidebar as shown here on the right.

`app/(kambaz)/courses/[cid]/Navigation.tsx`

```
import Link from "next/link";
export default function CourseNavigation() {
  return (
    <div id="wd-courses-navigation">
      <Link href="/courses/1234/home" id="wd-course-home-link">Home</Link><br/>
      <Link href="/courses/1234/modules" id="wd-course-modules-link">Modules
        </Link><br/>
      <Link href="/courses/1234/piazza" id="wd-course-piazza-link">Piazza</Link><br/>
      <Link href="/courses/1234/zoom" id="wd-course-zoom-link">Zoom</Link><br/>
      <Link href="/courses/1234/assignments" id="wd-course-assignments-link">
        Assignments</Link><br/>
      <Link href="/courses/1234/quizzes" id="wd-course-quizzes-link">Quizzes
        </Link><br/>
      <Link href="/courses/1234/grades" id="wd-course-grades-link">Grades</Link><br/>
      <Link href="/courses/1234/people/table" id="wd-course-people-link">People</Link><br/>
    </div>
  );
}
```

[Home](#)
[Modules](#)
[Piazza](#)
[Zoom](#)
[Assignments](#)
[Quizzes](#)
[People](#)

Then in the **Courses** component, use a **table** to display the sidebar on the left and the **Routes** on the right as shown below. For now, the **Routes** are just placeholders for several screens containing learning materials, assignments, grades, etc. Each screen will be implemented later in the chapter.

`app/(kambaz)/courses/[cid]/Layout.tsx`

```
import { ReactNode } from "react";
import CourseNavigation from "../Navigation";
export default async function CoursesLayout(
  { children, params }: Readonly<{ children: ReactNode; params: Promise<{ cid: string }> }>) {
  const { cid } = await params;
  return (
    <div id="wd-courses">
      <h2>Courses {cid}</h2>
      <hr />
      <table>
        <tbody>
          <tr>
            <td valign="top" width="200"> <CourseNavigation /> </td>
            <td valign="top" width="100%"> {children} </td>
          </tr>
        </tbody>
      </table>
    </div>
  );
}
```

1.4.5 Implementing the Kambaz Modules Screen

When a course is clicked in the **Dashboard**, the application navigates to the **Home** screen of that course by default. The **Home** screen displays a list of modules that contain a course's material. The screen shot below on the left illustrates the first module **Week 1, Lecture 1**, as well as providing links to **READING** material and **SLIDES**. This list of modules is the same content also available in the **Modules** shown below. Implement the **Modules** screen first and then reuse it to implement the **Home** screen. The screen consists of three columns containing the **Kambaz Navigation** sidebar in the first column, a **Course Navigation** sidebar in the second column, and the **Modules** in the third column. The **Kambaz** and **Course Navigation** sidebars are already implemented. Focus now on a prototype of the **Modules** as shown in Figure 1.4.5b. The **Modules** screen can be implemented as a set of nested lists where the top level list consists of a list of **modules**. Each module item can contain a nested list of **lessons**. Each lesson in turn can contain a list of different content items. The code sample below illustrates how nested lists could be used to implement the **Modules**.

app/(kambaz)/courses/[cid]/modules/page.tsx

```
export default function Modules() {
  return (
    <div>
      { /* Implement Collapse All button, View Progress button, etc. */ }
      <ul id="wd-modules">
        <li className="wd-module">
          <div className="wd-title">Week 1</div>
          <ul className="wd-lessons">
            <li className="wd-lesson">
              <span className="wd-title">LEARNING OBJECTIVES</span>
              <ul className="wd-content">
                <li className="wd-content-item">Introduction to the course</li>
                <li className="wd-content-item">Learn what is Web Development</li>
              </ul>
            </li>
          </ul>
        </li>
        <li className="wd-module"> <div className="wd-title">Week 2</div> </li>
        <li className="wd-module"> <div className="wd-title">Week 3</div> </li>
      </ul>
    </div>
  );
}
```

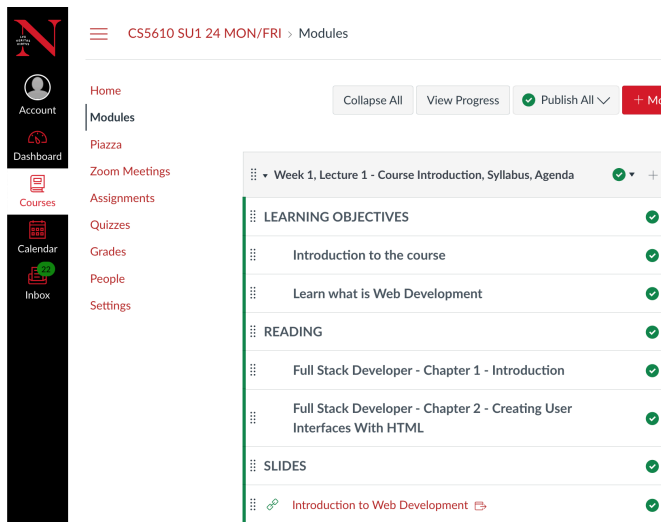


Figure 1.4.5a - The Modules Screen Screenshot

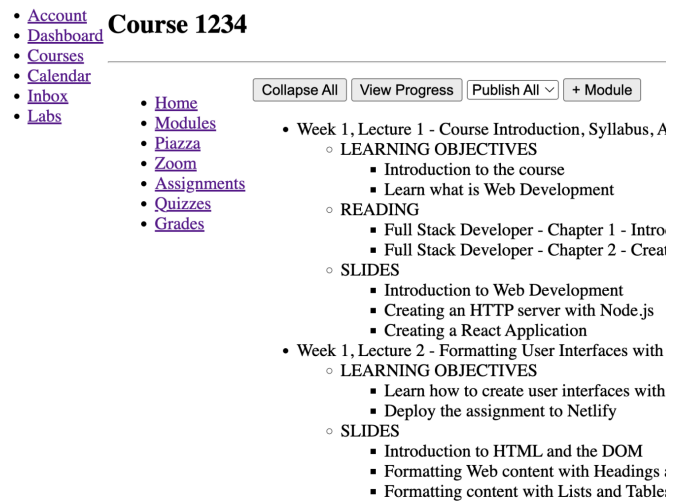


Figure 1.4.5b - Our implementation of the Modules Screen before styling

1.4.6 Implementing the Kambaz Course Home Screen

Clicking on a course in the **Dashboard** navigates to the course's **Home** screen shown below on the left. The **Home** screen contains four columns containing the **Kambaz** and **Courses Navigation** sidebars in the first and second column, the **Modules** in the third column, and a **Course Status** sidebar on the last column. The content for the first three columns has already been implemented. To implement the **Home** screen, implement the **Course Status** sidebar shown here on the right. Combine the sidebar with the other existing components to create the **Home** screen as shown in Figure 1.4.6b. Complete the **Course Status** sidebar on your own in a new file in `app/(kambaz)/courses/home/Status.tsx`.

`app/(kambaz)/courses/[cid]/home/Status.tsx`

```
export default function CourseStatus() {
  return (
    <div id="wd-course-status">
      <h2>Course Status</h2>
      <button>Unpublish</button> <button>Publish</button>
      { /* Complete on your own */ }
      <button>View Course Notifications</button>
    </div> );}
```

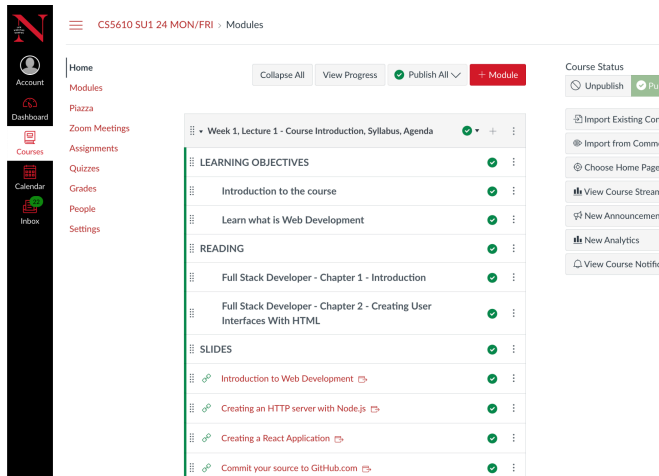


Figure 1.4.6a - The Home Screen Screenshot

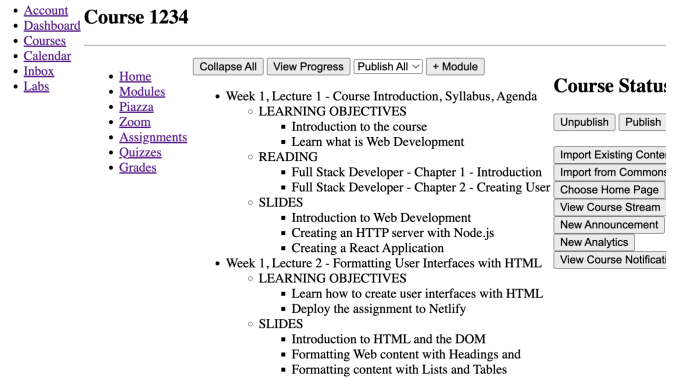


Figure 1.4.6b - Our implementation of the Home Screen before styling

Combine the **Course Status** with the **Modules** to create the **Home** screen as shown below.

`app/(kambaz)/courses/[cid]/home/page.tsx`

```
import Modules from "../modules/page";
import CourseStatus from "../Status";
export default function Home() {
  return (
    <div id="wd-home">
      <table>
        <tbody>
          <tr>
            <td valign="top" width="70%"> <Modules />
            <td valign="top"> <CourseStatus />
          </tr>
        </tbody>
      </table>
    </div>
  );
}
```

Replace the **Home** heading placeholder in the **Courses** screen with the actual **Home** component as shown below.

`app/(kambaz)/courses/[cid]/page.tsx`

```
import { redirect } from "next/navigation";

export default async function CoursesPage({ params, }: { params: Promise<{ cid: string }>; }) {
  const { cid } = await params;
  redirect(`/courses/${cid}/home`);
}
```

Confirm that navigating to a course from the dashboard displays the **Home** screen similar to the one shown below.

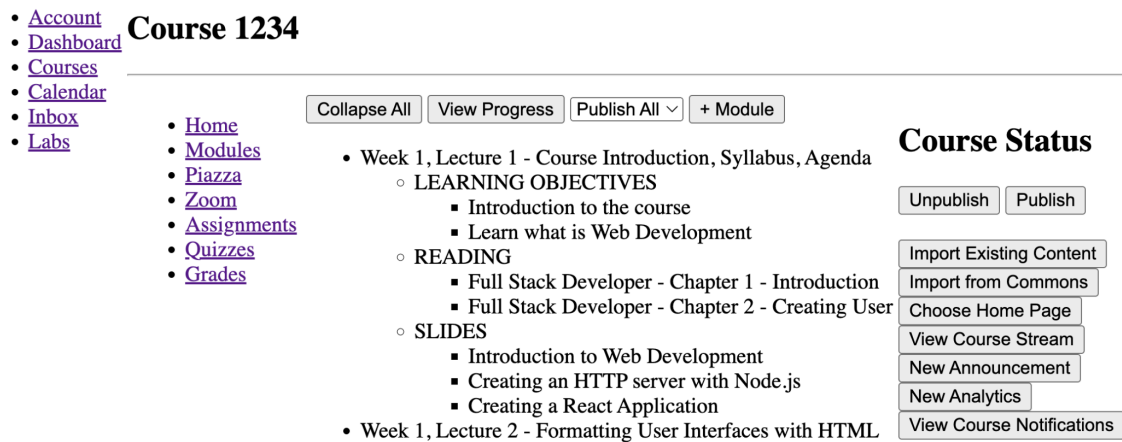


Figure 1.4.6c - The Home Screen

1.4.7 Assignments Screen (On Your Own)

So far this chapter has described how to create various **Kambaz** screens in detail. To confirm your skills, this section challenges you to implement a couple of screens on your own. Some sample code is provided as suggestions, but feel free to ignore the code and write your own.

The **Assignments Screen** lists all the assignments students are required to complete throughout a course. A screenshot of the assignments screen is shown below on the left. To navigate to the **Assignments Screen**, users navigate to the course from the **Dashboard**, and then click on the **Assignments** link in the **Course Navigation** sidebar. Assignments are grouped into categories such as **ASSIGNMENTS**, **QUIZZES**, **EXAMS**, and **PROJECT**, and are shown in the third column as shown below. Here only the **ASSIGNMENTS** category is shown and required. Implement a component called **Assignments** shown below in Figure 1.4.7b as a prototype of the **Assignments** screen in Figure 1.4.7a.

Feel free to use the following starter code to implement the **Assignments** component. If you prefer to build your own version from scratch, feel free to ignore the code provided. Be aware that if you decide to use the code provided, later chapters will build on the code from prior chapters. Your own implementation in earlier chapters might not be compatible with code in later chapters. Make the component look as shown below on the right. Make sure to keep the **id** and **className** attributes provided and all **li** (line items) in the **ul** (unordered list) must have the **className** set to **wd-assignment-list-item**.

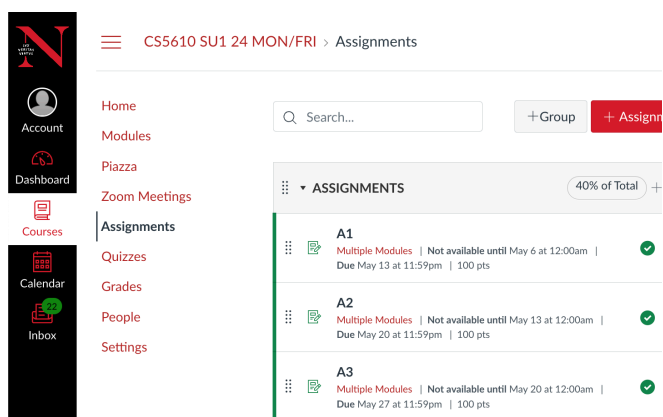


Figure 1.4.7a - The Assignments Screen Screenshot

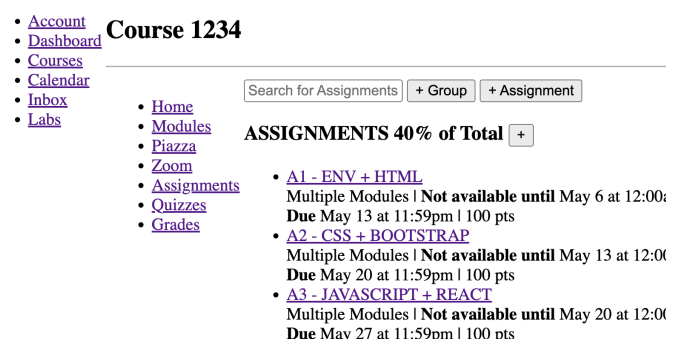


Figure 1.4.7b - Our implementation of the Assignments

`app/(kambaz)/courses/[cid]/assignments/page.tsx`

```
export default function Assignments() {
  return (
    <div id="wd-assignments">
      <input placeholder="Search for Assignments"
        id="wd-search-assignment" />
      <button id="wd-add-assignment-group">+ Group</button>
      <button id="wd-add-assignment">+ Assignment</button>
      <h3 id="wd-assignments-title">
        ASSIGNMENTS 40% of Total <button>+</button> </h3>
      <ul id="wd-assignment-list">
        <li className="wd-assignment-list-item">
          <Link href="/courses/1234/assignments/123"
            className="wd-assignment-link" >
            A1 - ENV + HTML
          </Link> </li>
        <li className="wd-assignment-list-item">
          {/* Complete On Your Own */}
        </li>
      </ul>
    </div>
  );
}
```

Search for Assignments + Group + Assignment

ASSIGNMENTS 40% of Total +

- [A1 - ENV + HTML](#)
Multiple Modules | **Not available until** May 6 at 12:00am |
Due May 13 at 11:59pm | 100 pts
- [A2 - CSS + BOOTSTRAP](#)
Multiple Modules | **Not available until** May 13 at 12:00am |
Due May 20 at 11:59pm | 100 pts
- [A3 - JAVASCRIPT + REACT](#)
Multiple Modules | **Not available until** May 20 at 12:00am |
Due May 27 at 11:59pm | 100 pts

1.4.8 Assignment Editor Screen (On Your Own)

When clicking the title of an assignment in the **Assignments** screen, **Kambaz** navigates to the **Assignment Editor** screen where faculty can edit the assignment's details such as the title, points, and due date as shown below on the left. In `app/(kambaz)e/courses/[cid]/assignments/[aid]/page.tsx` create the **AssignmentEditor** component that implements a prototype of the **Assignment Editor** screen as shown below on Figure 1.4.8b. Feel free to use the following starter code to get going with the implementation of the **Assignment Editor** screen.

`app/(kambaz)/courses/[cid]/assignments/[aid]/page.tsx`

```
export default function AssignmentEditor() {
  return (
    <div id="wd-assignments-editor">
      <label htmlFor="wd-name">Assignment Name</label> {/* clicking this label */}
      <input id="wd-name" defaultValue="A1 - ENV + HTML" /> <br /> {/* selects this field */}
      <textarea id="wd-description">
        The assignment is available online Submit a link to the landing page of
      </textarea>
      <br />
      <table>
        <tr>
          <td align="right" valign="top">
            <label htmlFor="wd-points">Points</label> {/* clicking this label */}
          </td>
          <td>
            <input id="wd-points" defaultValue={100} /> {/* selects this field */}
          </td>
        </tr>
      </table>
      {/* Complete on your own */}
    </div>
  );
}
```

Make sure that clicking the labels above or next to a text fields, gives focus to the field. Clicking the labels next to a checkbox, toggles the checkbox. Clicking the label above a date input field, gives focus to the date input field. That is, clicking the label or text next to the corresponding field should act on that field. In the **Courses** component, add a **Route** to navigate to the new **Assignment Editor** screen when clicking the title of an assignment. For now all assignments navigate to the same **AssignmentEditor** component and display the same

information for all assignments. Later chapters will discuss how to retrieve information for a specific assignment and display the corresponding information.

Figure 1.4.8a - The Assignment Editor Screen Screenshot

Figure 1.4.8b - Our implementation of the Assignment Editor Screen before styling

1.5 Committing Code to Source Control

So far the Web application is running on your local development environment, only accessible to you. To make it available to everyone on the Web, the application needs to be deployed to a remote public server that will host and run the application, serving the content to anyone who knows the URL of the servers where the application is running. To deploy the application, first the source code needs to be committed and pushed to a source repository such as GitHub. This section demonstrates how to share the source code to a public GitHub repository and then deploy it to **Vercel**, a popular service for hosting **Next.js React** Web applications.

1.5.1 Install a Git Client

Git is a popular source control software developers use to share and collaborate on projects. On macOS you already get a command line git client. You can fully interact with **github.com** from the terminal or **Visual Studio**

Code. On Windows OS, you'll need to install a git client from where you will be able to issue the same commands from a console. Download git for windows from <https://git-scm.com/download/win>, run the installer and follow the instructions. At the end of the installation you should be able to execute git commands from new console instances. All examples in this course assumes you have git installed in your OS.

You can also install a graphical git client if you prefer. There are a lot of alternatives, but the author suggests **Sourcetree** since it works well and consistently in both macOS and Windows. To install Sourcetree download from <https://www.sourcetreeapp.com/>, install, and follow instructions. We will not be covering how to use Sourcetree, but you are free to use it if you wish. All examples in this course will be using the command line git client.

1.5.2 Ignoring Files and Directories

Git can keep track of all the files in your project, but there are some files or directories that you don't want to keep track of, for instance compiled classes, libraries, etc. You can configure which files and directories you want git to ignore by listing them in a file called **.gitignore** at the root of your project, which should already exist. Create the file if it does not already exist. With a text editor or from IntelliJ, edit the file **.gitignore**. Towards the top of the file, in a blank line, type the following. **Note** the period at the beginning of the file!!

```
.gitignore
.idea
node_modules
```

This tells git that the **.idea** folder should be ignored because it is a directory specific to IntelliJ and not relevant to React project itself. If you are using other IDEs, then you might want to add other files or directories here relevant to your specific IDE. The **node_modules** folder should also be ignored since it contains library files that should not be added to source control. **Note:** make sure that IDE specific folders and the **node_modules** folders don't make it into the repository!!

1.5.3 Creating a Remote Source Repository

If you don't already have an account on **github.com**, create a new account or use an account you already have at **github.com**. Do not use the university's github or your work's source control if you already have one. Login to your **github.com** account and click the **New** button to create a new **public repository** called **kambaz-next-js**, just like the name of the React project you created earlier. Here's an example on how it looks on my laptop. Your username "**jannunzi**" will obviously be different.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Owner *



Repository name *

/ kambaz-next-js

✓ kambaz-next-js is available.

Once you create the remote repository on github, it will display commands on how to commit and push your code from your computer up to the remote repository. The commands will be similar to the ones shown below. The username "**jannunzi**" will obviously be different.

Quick setup — if you've done this kind of thing before

 Set up in Desktop

or

HTTPS

SSH

`https://github.com/jannunzi/kambaz-next-js.git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# kambaz-next-js" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/jannunzi/kambaz-next-js.git
git push -u origin main
```

1.5.4 Adding and Committing Code to a Remote Repository

Now that we have a remote source repository, let's add, commit and push our code to the repository. From the terminal or console, make sure you are in the **kambaz-next-js** directory and then type the following commands which are based on the commands git suggested. Ignore the commentary on the right. Also, your actual URL below will differ for you, so **don't blindly copy the example below**. Use the commands git suggested for you.

```
git init                                # initializes local repository
git add .                               # adds all files to repository
git commit -m "first commit"            # commits files with message
git remote add origin https://github.com/jannunzi/kambaz-next-js.git # adds remote repository
git push -u origin main                  # copies local files to remote
```

Refresh the remote repository on github.com and confirm that the files are now available there

1.5.5 Using Personal Access Tokens

While pushing to GitHub you might encounter an error stating that **password authentication was removed on August 13, 2021**. One way to fix this is to generate a **personal access token** and use that instead of your password. To generate a personal access token go to your GitHub **Settings**, and then **Developer Settings**. Click on **Personal access tokens** and then on **Generate new token**. Enter a short description in the **Note** field, select **No expiration** for **Expiration**, and grant all access privileges by selecting all the checkboxes under **Select scopes**. You are welcome to be more restrictive if you want. Click on **Generate token** and copy the long unique access token to your clipboard. Note that this will be the only opportunity to copy the token and you fail to do so you'll have to delete this token, create a brand new one, and try again. With the token copied to the clipboard, try pushing again to GitHub, but this time paste the token when asked for a password.

If you are not being asked for a password then GitHub might be using a cached authentication. To clear cached GitHub authentications on Windows go to the **Credential Manager**, click on **Windows Credentials**, click the GitHub credentials, and click **Remove**. This time github should ask for your username and password again when trying to push. Paste the access token when asked for the password. To clear cached authentications on macOS, go to your **Key Chain** and search for github. Remove the GitHub key chain.

1.6 Deploying Next.js Projects to the Web

Create an account at <https://vercel.com> if you don't already have one. Choose your preferred authentication method, e.g., Google, GitHub, or email verifying your identify via email, phone or text. Import the Git repository created earlier, authorizing Vercel to interact with your GitHub repositories. In the **Import Git Repository** dialog, search for the repository by name and click **Import**. In the **New Project** dialog verify the **Project Name** is the same as the name in your repo, and the **Framework Preset** is **Next.js**, then click **Deploy**. In the **Deployment** dialog, open the **Build Logs** so you can follow along how Vercel attempts to deploy your application, paying attention to cloning the GitHub project, building, and deploying. If any errors are listed in the logs, fix them in your code, commit, push again, and a new Vercel deployment will kick off automatically. Fix all bugs in the log until there's a successful deployment and you see a **Congratulations** message. Click on the **Continue to Dashboard** to see the **Production Deployment** in the **Overview** tab. Under the **Domains** label, click the URL to view your application running on Vercel. To see all your failed and successful deployments, click the **Deployments** tab which lists your last deployment at the top. Navigate to your latest deployment by clicking it and notice the **Status**, **Domains**, **Source**, **Deployment Configuration**, and **Build Logs**. The **Domains** section contains several URLs of which we are only interested in the first and it's this URL which you will submit for this assignment. All URLs navigate to the same deployed application, but the URLs are different capturing useful information regarding the **author**, **branch**, and **commit**. The URLs are protected so that they can only be accessed by authorized users. In the Deployment Settings, disable the protection so that TAs and instructors can easily access your Web application. To disable URL protection go to the **Deployment** tab of your project, expand the **Deployment Settings** and navigate to **Deployment Protection Standard Protection** as shown below.

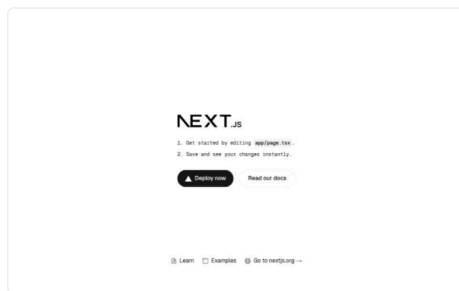
In the **Project Settings** screen, in **Deployment Protection** section, under the **Vercel Authentication**, uncheck the toggle and save as shown below. Now TAs will be able to navigate to the URL you share when you submit the assignment without having to be logged in and part of your project team.

Deployment Details

Share

Visit

...



Created

jannunzi Sep 5

Status

Ready Stale

Duration

43s 88d ago

Environment

Production **Current**

Domains

kambaz-next-js-cs5610-fa25-05.vercel.app +1

kambaz-next-js-cs5610-fa25-05-git-main-jannunzis-projects.vercel.app

kambaz-next-js-cs5610-fa25-05-ocr1y3iv2-jannunzis-projects.vercel.app

Source

main

216a71c Initial commit from Create Next App

Deployment Settings

Build Settings

On-Demand Concurrent Builds

✖ Disabled

Build Machine

Standard performance 4 vCPUs 8 GB Memory

Prioritize Production Builds

✔ Enabled

Runtime Settings

Fluid Compute

✔ Enabled

Function CPU

Standard 1 vCPU 2 GB Memory

Deployment Protection

✔ Standard Protection

Skew Protection

✖ Disabled

Cold Start Prevention

✖ Disabled

Project Settings

Search...

General

Build and Deployment

Domains

Deployment Protection

Functions

Vercel Authentication

Ensures visitors to your deployments are logged in to Vercel and are a member of your team.



Enabled for

Standard Protection

Learn more about [Vercel Authentication](#)

Save

1.7 Conclusion

By the end of this chapter, you'll be expected to have completed the following tasks.

1. Downloaded and install the latest **Node.js** as described.
2. Created a React application called **kambaz-next-js** as described.
3. Completed all the lab exercises.
4. Prototyped the Kambaz React user interface with HTML and React.js pages and components.
5. Pushed the source code of the React application **kambaz-next-js** to a remote source repository in **GitHub.com** as described.
6. Confirm the **labs/page.tsx** contains a **TOC.tsx** that references each of the labs and Kambaz. Add a link to your repository in GitHub. The link should have an **ID** attribute with a value of **wd-github**.

7. In ***labs/page.tsx***, add your full name: first name first and last name second. Use the same name as in Canvas.
8. Deployed the ***kambaz-next-js*** React application to ***Vercel*** as described.
9. As a deliverable in ***Canvas***, submit:
 - a. The URL to your React application running on Vercel
 - b. The URL to your Git repository